

AI Based Focus and Attention Analysis System

A Minor Project Report

Submitted in partial fulfillment of requirement of the

Degree of

**BACHELOR OF TECHNOLOGY in COMPUTER
SCIENCE & ENGINEERING**

BY

Amritansh Jain EN23CS301122

Ankit Lovanshi EN23CS301137

Ansh Gupta EN23CS301150

Under the Guidance of

Prof. Deepika Saraswat

Prof. Jyotsana Goyal



Department of Computer Science & Engineering

Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE- 453331

APRIL-2026

Report Approval

The project work “**AI Based Focus and Attention Analysis System**” is hereby approved as a creditable study of an engineering subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn there in; but approve the “Project Report” only for the purpose for which it has been submitted.

Internal Examiner

Name:

Designation

Affiliation

External Examiner

Name:

Designation

Affiliation

Declaration

We hereby declare that the project entitled “**AI Based Focus and Attention Analysis System**” submitted in partial fulfillment for the award of the degree of Bachelor of Technology in ‘Computer Science Engineering’ completed under the supervision of **Prof. Deepika Saraswat, Prof. Jyotsana Goyal** Faculty of Engineering, Medicaps University Indore is an authentic work.

Further, we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Signature and name of the students

1. Amritansh Jain (EN23CS301122)

Signature : _____

2. Ankit Lovanshi (EN23CS301137)

Signature : _____

3. Ansh Gupta (EN23CS301150)

Signature : _____

Date : _____

Certificate

We certify that the project entitled “**AI Based Focus and Attention Analysis System**” submitted in partial fulfillment for the award of the degree of Bachelor of Technology by Amritansh Jain (EN23CS301122), Ankit Lovanshi (EN23CS301137), Ansh Gupta (EN23CS301150) is the record carried out by them under our guidance and that the work has not formed the basis of award of any other degree elsewhere.

Prof. Deepika Saraswat,

Prof. Jyotsana Goyal

Department of Computer Science and
Engineering

Medicaps University, Indore

Dr. Kailash C. Bandhu

Head of the Department

Computer Science & Engineering

Medicaps University, Indore

Acknowledgements

We would like to express my deepest gratitude to the Honorable Chancellor, **Shri R C Mittal**, who has provided me with every facility to successfully carry out this project, and my profound indebtedness to **Dr. D. K. Patnaik**, Vice Chancellor, Medicaps University, whose unfailing support and enthusiasm has always boosted up my morale. I also thank **Dr. Ratnesh Litoriya** Associate Dean (Computing and Allied Branches) Medicaps University, for giving me a chance to work on this project. I would also like to thank my Head of the Department **Dr. Kailash Chandra Bandhu** for his continuous encouragement for the betterment of the project.

It is their help and support, due to which we became able to complete the design and technical report.

Without their support this report would not have been possible.

Amritansh Jain (EN23CS301122)

Ankit Lovanshi (EN23CS301137)

Ansh Gupta (EN23CS301150)

B.Tech. III Year , CSE – C

Department of Computer Science & Engineering

Faculty of Engineering

Medicaps University, Indore

Abstract

The increasing presence of digital distractions makes it difficult for students to maintain effective focus during study sessions. This project presents an AI Based Focus and Attention Analysis System, an offline application that analyzes attention using computer vision techniques. The system captures video through a webcam, detects facial landmarks using MediaPipe, and extracts signals such as blink rate, eye-closure duration, and face presence. A rule-based model processes these signals to generate a focus score between 0 and 100. The application is developed in Python with a Tkinter-based interface for real-time monitoring, while session data is stored locally in JSON format to ensure privacy. The system provides a simple and efficient method to evaluate attention and improve study productivity without relying on complex machine learning models.

Keywords : Computer Vision, Focus Analysis, Attention Monitoring, Facial Landmark Detection

Table of Contents

Chapter	Title	Page No.
	Report Approval	ii
	Declaration	iii
	Certificate	iv
	Acknowledgement	v
	Abstract	vi
	Table of Contents	vii
	List of figures	ix
	List of tables	ix
Chapter 1	Introduction	
	1.1 Introduction	10
	1.2 Literature Review	11
	1.3 Objectives	12
	1.4 Significance	12
	1.5 Research Design	13
	1.6 Source of Data	14
	1.7 Chapter Scheme	15
Chapter 2	Requirements specification	
	2.1 User Characteristics	17
	2.2 Functional Requirements	17
	2.3 Dependencies	18
	2.4 Performance Requirements	19
	2.5 Hardware Requirements	20
	2.6 Constraints & Assumptions	20
Chapter 3	Design	
	3.1 Algorithm	22
	3.2 System Architecture	23
	3.3 System Design	24

	3.3.1 Data Flow Diagrams (Level 0, Level 1)	24
	3.3.2 Activity Diagram	26
	3.3.3 Flow Chart	27
	3.3.4 Class diagram	28
	3.3.5 Sequence Diagram	28
Chapter 4	Implementation, Testing, and Maintenance	
	4.1 Introduction to Languages, IDE's, Tools and Technologies used for Implementation	31
	4.2 Testing Techniques and Test Plans (According to project)	32
	4.3 Installation Instructions	33
	4.4 End User Instructions	34
Chapter 5	Results and Discussions	
	5.1 User Interface Representation	36
	5.2 Brief Description of Various Modules of the system	37
	5.3 Snapshots of system	38
	5.4 Back End Representation	39
	5.5 Snapshots of Database Tables with Brief Description	40
Chapter 6	Summary and Conclusions	41
Chapter 7	Future scope	42
	Bibliography	43

List of Figures

Figure No.	Caption	Page No.
3.3.1	Data flow diagrams	23
3.3.2	Activity diagram	25
3.3.3	Flowchart	26
3.3.4	Class diagram	27
3.3.5	Sequence diagram	27
5.1	GUI snapshot	34
5.3.1	Initial interface	36
5.3.2	Session in progress	36
5.3.3	Focus display score	36
5.3.4	Session complete	37
5.5.1	JSON data storage	38

List of Tables

Table No.	Title	Page No.
2.5	Hardware requirements	19

Chapter 1

1.1 Introduction

In the modern digital era, students are increasingly exposed to multiple sources of distraction, such as social media, multitasking environments, and continuous screen exposure. Although students may spend considerable time studying, they often lack awareness of their actual level of focus and attention during these sessions. Traditional productivity tools primarily measure the duration of study rather than the quality of attention, making them insufficient for evaluating effective learning.

Focus and attention are critical factors in academic performance and cognitive engagement. However, measuring attention objectively is a complex task, as it involves analyzing behavioral patterns rather than relying solely on self-assessment. There is a growing need for systems that can provide real-time insights into attention levels without being intrusive or dependent on external data sources.

This project, titled “AI Based Focus and Attention Analysis System,” aims to address this challenge by developing an offline desktop application that analyzes student focus using computer vision techniques. The system captures real-time video input through a webcam and processes facial and eye-related features to estimate attention levels. Key behavioral indicators such as face presence, blink frequency, and eye-closure duration are used to derive a focus confidence score.

The system is designed using Python and integrates OpenCV for video processing and MediaPipe for facial landmark detection. A rule-based analysis model is applied to convert extracted signals into a meaningful focus score ranging from 0 to 100. The results are displayed through a user-friendly graphical interface built using Tkinter, allowing users to monitor their focus during study sessions.

The primary objective of this system is to provide a privacy-preserving, explainable, and efficient method for assessing attention without relying on complex machine learning models or cloud-based processing. By offering real-time feedback, the system helps students become more aware of their study behavior and encourages improved concentration and productivity.

1.2 Literature Review

OpenCV is a widely used open-source computer vision library that provides efficient tools for real-time image and video processing. It supports various functionalities such as face detection, object recognition, and frame analysis, making it suitable for applications involving live video input. Its ability to process frames quickly and accurately makes it an essential component for systems that rely on continuous visual data analysis. [1]

MediaPipe provides a robust framework for real-time face and landmark detection using pre-trained models. It enables precise detection of facial features, including eye regions, which are crucial for analyzing attention-related behavior such as blinking and eye closure. Its lightweight design allows it to run efficiently on local systems without requiring high computational resources. [2]

Python is a versatile and widely adopted programming language known for its simplicity and extensive ecosystem of libraries. It supports rapid development and integration of various components, making it highly suitable for building applications that combine computer vision, data processing, and graphical interfaces. [3]

NumPy is a fundamental library for numerical computing in Python, offering efficient handling of arrays and mathematical operations. It plays a key role in processing and analyzing numerical data derived from visual signals, such as calculating ratios and applying mathematical transformations required for attention analysis. [4]

Tkinter is a standard Python library used for developing graphical user interfaces. It provides simple and effective tools to create interactive desktop applications, allowing users to control system operations and view outputs such as focus scores and session durations in real time. [5]

Rosebrock demonstrated practical implementations of computer vision techniques using Python and OpenCV, highlighting how real-time systems can be developed efficiently. His work emphasizes the application of image processing techniques in detecting and analyzing visual features, which is relevant for systems that monitor user behavior. [6]

Szeliski provided a comprehensive overview of computer vision algorithms and their applications in understanding visual data. His work explains the theoretical foundations of image analysis,

feature extraction, and pattern recognition, which are essential for developing systems that interpret human attention through visual cues. [7]

Bradski and Kaehler explored the use of OpenCV for building computer vision applications, focusing on feature detection and image processing techniques. Their work demonstrates how vision-based systems can be implemented effectively using standard libraries, making them accessible for real-time applications. [8]

1.3 Objectives

The main objectives of the project are as follows:

1. To develop an offline system that analyzes student focus and attention during study sessions.
2. To utilize computer vision techniques for detecting facial and eye-related behavioral signals in real time.
3. To measure key attention indicators such as face presence, blink frequency, and eye-closure duration.
4. To design a rule-based model that converts extracted signals into a meaningful focus score (0–100).
5. To provide real-time feedback to users through a graphical user interface.
6. To ensure user privacy by avoiding data storage of images or reliance on cloud-based processing.
7. To create a simple, efficient, and explainable system that does not depend on complex machine learning models.
8. To help users improve study habits and productivity by increasing awareness of their attention patterns.

1.4 Significance

The significance of the proposed system lies in its ability to provide a practical and efficient solution for analyzing student focus in real time. In today's academic environment, students often face multiple distractions, and there is a lack of reliable tools to measure their actual level of attention during study sessions. This system addresses this gap by introducing a non-intrusive and privacy-preserving approach using computer vision techniques. Unlike traditional systems that

rely on complex machine learning models or external data storage, the proposed system operates entirely offline and does not store sensitive visual data, ensuring user privacy and security. It shifts the focus from measuring study time to evaluating study quality by providing a real-time focus score based on observable behavioral signals. This helps users become more aware of their attention patterns, leading to improved concentration, better study habits, and enhanced productivity. Additionally, the use of lightweight technologies makes the system accessible and easy to implement, while the rule-based approach ensures transparency and explainability. Overall, the project demonstrates the effective application of simple AI techniques in improving human behavior and performance.

1.5 Research Design

The research design for this project focuses on developing a systematic and structured approach to analyze student attention using computer vision techniques.

The design is based on a real-time observational approach, where user behavior is continuously monitored through a webcam without storing any visual data. The system follows a modular architecture, ensuring that each component performs a specific function independently.

The key stages of the research design are as follows:

Data Acquisition:

Real-time video frames are captured using a webcam through OpenCV.

Feature Extraction:

Facial landmarks are detected using MediaPipe to identify key regions such as eyes and face position.

Signal Processing:

Behavioral signals such as blink count, eye-closure duration, and face presence time are extracted from the detected landmarks.

Normalization:

Extracted signals are converted into standardized ratios to ensure consistency across different session durations.

Focus Estimation:

A rule-based model is applied to calculate a focus score based on presence and penalty mechanisms.

Output Generation:

The computed focus score is displayed through a graphical user interface in real time.

Data Storage:

Session results are stored locally in JSON format for future reference.

This research design ensures that the system remains efficient, privacy-preserving, and easy to implement while providing reliable insights into user attention.

1.6 Source of Data

The system primarily relies on real-time data captured directly from the user through a webcam. Unlike traditional data-driven systems that require large pre-collected datasets, this project utilizes live input for analysis.

The sources of data used in the system are as follows:

Primary Data Source:

Real-time video frames captured using a webcam serve as the primary source of data. These frames are processed continuously to extract facial and eye-related features.

Derived Data:

Behavioral signals such as blink count, eye-closure duration, and face presence time are derived from the captured video frames using computer vision techniques.

Local Data Storage:

Processed results, including session duration and focus scores, are stored locally in JSON format for analysis and reference.

Libraries and Frameworks:

Pre-built models and tools from libraries such as MediaPipe are used to detect facial landmarks, eliminating the need for external training datasets.

This approach ensures that the system operates independently without relying on external or cloud-based data sources, thereby maintaining user privacy and reducing system complexity.

1.7 Chapter Scheme

The project report is organized into several chapters to provide a clear and systematic understanding of the system design, implementation, and results.

Chapter 1: Introduction

This chapter provides an overview of the project, including the introduction, literature review, objectives, significance, research design, and sources of data.

Chapter 2: Requirements Specification

This chapter describes the system requirements, including user characteristics, functional and non-functional requirements, dependencies, performance requirements, hardware requirements, and system constraints.

Chapter 3: Design

This chapter explains the overall system design, including algorithms, data flow diagrams, activity diagrams, flowcharts, class diagrams, and other relevant design representations.

Chapter 4: Implementation, Testing, and Maintenance

This chapter covers the technologies used for implementation, testing methodologies, installation process, and user instructions for operating the system.

Chapter 5: Results and Discussions

This chapter presents the system outputs, graphical user interface, module descriptions, and snapshots of the system along with explanations.

Chapter 6: Summary and Conclusions

This chapter summarizes the overall work, findings, and conclusions drawn from the project.

Chapter 7: Future Scope

This chapter discusses possible improvements and extensions that can be implemented in the future.

Appendix and Bibliography

These sections include additional supporting material, references, and sources used in the project.

Chapter 2

2.1 User Characteristics

The system is designed for users who require a simple and effective method to monitor their focus during study or work sessions. The primary users of this system are students, but it can also be used by individuals involved in tasks that require sustained attention.

The key user characteristics are as follows:

Basic Computer Knowledge:

Users should have a basic understanding of operating a computer system, including launching applications and interacting with graphical interfaces.

Familiarity with Desktop Applications:

Users are expected to be comfortable using simple desktop-based software with buttons and display elements.

No Technical Expertise Required:

The system is designed to be user-friendly and does not require any prior knowledge of artificial intelligence, computer vision, or programming.

Access to Required Hardware:

Users must have a device equipped with a webcam, as the system relies on real-time video input for analysis.

Focus on Academic or Productive Tasks:

The system is primarily intended for students or individuals who wish to improve their concentration and productivity during study or work sessions.

Awareness of Privacy Considerations:

Users should understand that the system processes video input locally and does not store or transmit personal data, ensuring privacy.

Overall, the system is designed to be accessible to a wide range of users with minimal technical barriers, making it suitable for everyday use in educational and personal environments.

2.2 Functional Requirements

The functional requirements describe the core features and operations that the system must perform. The system should fulfill the following functions:

User Session Control:

The system should allow the user to start and stop a study session using the graphical user interface.

Real-Time Video Capture:

The system should capture live video input from the webcam continuously during the session.

Face Detection:

The system should detect the presence of a user's face in the video frame using computer vision techniques.

Facial Landmark Detection:

The system should identify facial landmarks, particularly around the eyes, using MediaPipe.

Blink Detection:

The system should detect and count the number of eye blinks during the session.

Eye Closure Detection:

The system should measure the duration for which the eyes remain closed.

Face Presence Tracking:

The system should calculate the duration for which the user's face is visible during the session.

Focus Score Calculation:

The system should compute a focus score based on the extracted signals using a rule-based model.

Real-Time Display:

The system should display the focus score and session duration in real time through the graphical interface.

Session Data Storage:

The system should store session results such as duration and focus score in a local file (JSON format).

User-Friendly Interface:

The system should provide a simple and easy-to-use interface for interaction.

2.3 Dependencies

The system depends on several software libraries, hardware components, and environmental conditions for proper functioning. The key dependencies are as follows:

Programming Language:

Python is required as the core programming language for implementing the system.

OpenCV Library:

Used for capturing and processing real-time video frames from the webcam.

MediaPipe Library:

Required for detecting facial landmarks and extracting eye and face-related features.

NumPy Library:

Used for performing numerical and mathematical operations during signal processing.

Tkinter Library:

Used for designing and managing the graphical user interface.

Operating System Compatibility:

The system requires a compatible operating system such as Windows, Linux, or macOS.

Webcam Availability:

A functional webcam is necessary for capturing real-time video input.

Python Environment:

Proper installation of Python and required libraries using pip is necessary.

Local File System:

Required for storing session data in JSON format.

These dependencies must be satisfied for the system to function correctly and deliver accurate results.

2.4 Performance Requirements

The system is expected to perform efficiently in real-time while maintaining accuracy and responsiveness. The key performance requirements are as follows:

Real-Time Processing:

The system should process video frames continuously and provide focus analysis without noticeable delay.

Low Latency:

The response time for updating the focus score and session data should be minimal to ensure smooth user experience.

Efficient Resource Utilization:

The system should operate using moderate CPU and memory resources without requiring high-end hardware.

Accuracy of Detection:

The system should accurately detect facial landmarks, blinks, and eye closure to ensure reliable focus estimation.

Stability:

The system should run continuously without crashes or interruptions during a session.

Scalability (Basic Level):

The system should handle varying session durations without degradation in performance.

Responsiveness of User Interface:

The graphical interface should remain responsive while background processing is ongoing.

Offline Functionality:

The system should function completely offline without dependency on internet connectivity.

These performance requirements ensure that the system provides a smooth, reliable, and efficient user experience.

2.5 Hardware Requirements

Component	Requirement
Processor (CPU)	Minimum Intel i3 or equivalent processor recommended for smooth execution
RAM	At least 4 GB RAM required for proper real-time processing
Webcam	Functional webcam required for capturing real-time video input
Storage	Minimum 500 MB free disk space for project files and data storage
Display	Monitor with at least 1024×768 resolution for proper GUI display
Input Devices	Keyboard and mouse for system interaction

2.6 Constraints & Assumptions

The system operates under certain limitations and predefined assumptions that influence its performance and applicability.

Constraints

Lighting Conditions:

The system's accuracy depends on proper lighting; poor lighting may affect face and eye detection.

Camera Quality:

Low-resolution or faulty webcams can reduce detection accuracy.

Single User Focus:

The system is designed for one user at a time and may not perform correctly with multiple faces in the frame.

Limited Behavioral Signals:

The system uses a limited set of indicators (blink rate, eye closure, face presence) and does not analyze deeper cognitive or emotional states.

No Deep Learning Models:

The system avoids complex machine learning models, which may limit advanced prediction capabilities.

Desktop-Based Application:

The system is designed for desktop use and may not be optimized for mobile devices.

Assumptions**User Position:**

The user is assumed to be seated in front of the camera at a reasonable distance.

Stable Environment:

The system assumes minimal background movement or interference.

Continuous Visibility:

The user's face remains visible for most of the session.

Proper Camera Alignment:

The webcam is positioned correctly to capture the user's face clearly.

User Cooperation:

The user interacts with the system as intended (starting/stopping sessions properly).

These constraints and assumptions define the operational boundaries of the system and help ensure consistent performance under normal conditions.

Chapter 3

3.1 Algorithm

The system follows a structured algorithm to analyze user focus using real-time video input and behavioral signal processing. The algorithm is designed to be simple, efficient, and suitable for real-time execution.

Algorithm Steps

Step 1: Start Session

The user initiates the session using the graphical interface.

Step 2: Capture Video Frames

The system captures continuous video frames from the webcam using OpenCV.

Step 3: Detect Face and Landmarks

MediaPipe is used to detect facial landmarks, focusing on eye and face regions.

Step 4: Extract Behavioral Signals

From the detected landmarks, the system calculates:

- i. Blink count
- ii. Eye-closure duration
- iii. Face presence time

Step 5: Normalize Signals

The extracted values are normalized with respect to the total session time.

Step 6: Calculate Base Focus Score

Base focus is calculated using face presence:

$$Base\ Score = \left(\frac{Face\ Time}{Total\ Time} \right) \times 100$$

Step 7: Apply Blink Penalty

Blink rate is calculated per minute and penalty is applied:

$\leq 20 \rightarrow 0$ penalty

21–30 $\rightarrow 5$ penalty

30 $\rightarrow 10$ penalty

Step 8: Apply Eye Closure Penalty

Eye-closure ratio is calculated and penalty is applied:

$\leq 5\% \rightarrow 0$ penalty

5–15% $\rightarrow 10$ penalty

15% $\rightarrow 20$ penalty

Step 9: Compute Final Focus Score

$$Final\ Score = Base\ Score - (Blink\ Penalty + Eye\ Closure\ Penalty)$$

Step 10: Clamp Score

The final score is limited between 0 and 100.

Step 11: Display Results

The focus score and session duration are displayed on the GUI.

Step 12: Store Session Data

The session data is saved locally in JSON format.

Step 13: End Session

The session ends when the user stops it.

3.2 Function Oriented Design

The system follows a function-oriented design approach, where the overall functionality is divided into smaller, well-defined modules. Each module performs a specific task, ensuring modularity, ease of development, and maintainability.

The major functional components of the system are as follows:

Video Capture Module

Responsible for capturing real-time video frames from the webcam using OpenCV.

Face and Landmark Detection Module

Uses MediaPipe to detect the user's face and extract facial landmarks, especially around the eye region.

Signal Extraction Module

Processes the detected landmarks to extract behavioral signals such as:

- i. Blink count
- ii. Eye-closure duration
- iii. Face presence time

Focus Analysis Module

Implements the rule-based logic to:

- i. Normalize signals
- ii. Calculate base focus score
- iii. Apply blink and eye-closure penalties
- iv. Compute final focus score

User Interface Module

Developed using Tkinter, this module:

- i. Allows users to start and stop sessions
- ii. Displays real-time focus score and session duration

Data Storage Module

Stores session-related data such as focus score and duration in JSON format for future reference.

Interaction Between Modules

The system operates by passing data sequentially between modules:

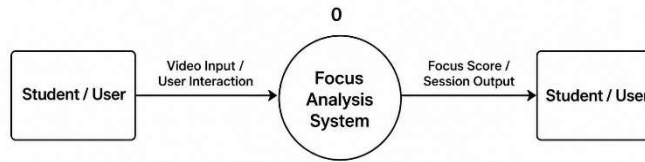
Video Capture → Landmark Detection → Signal Extraction → Focus Analysis → UI Display → Data Storage

Each module works independently but communicates through well-defined inputs and outputs, ensuring a clear separation of concerns.

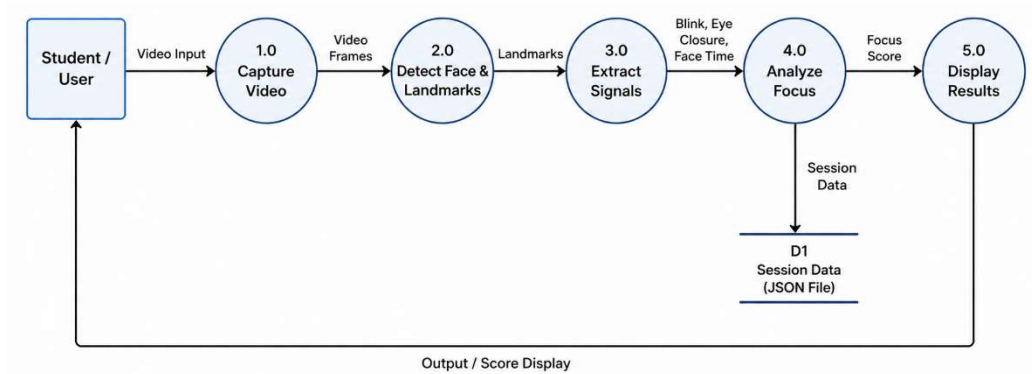
3.3 System Design

3.3.1 Data Flow Diagrams

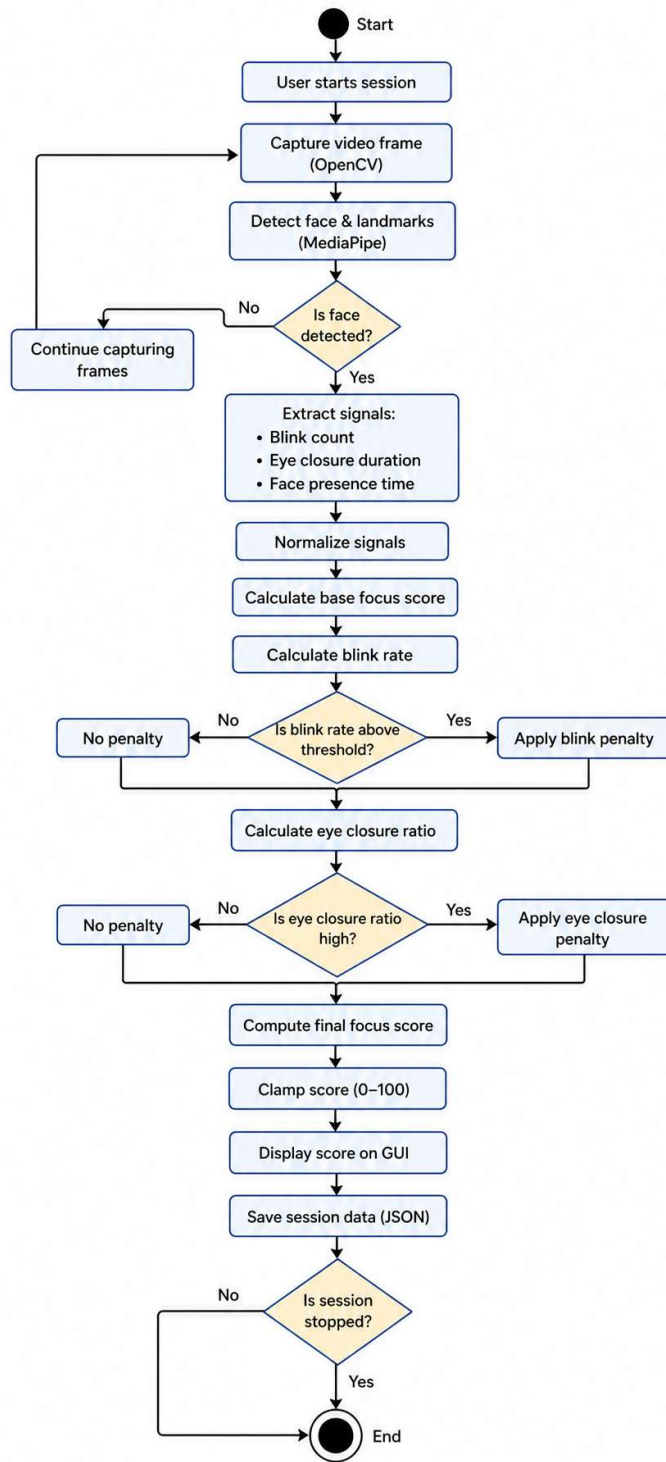
Level 0



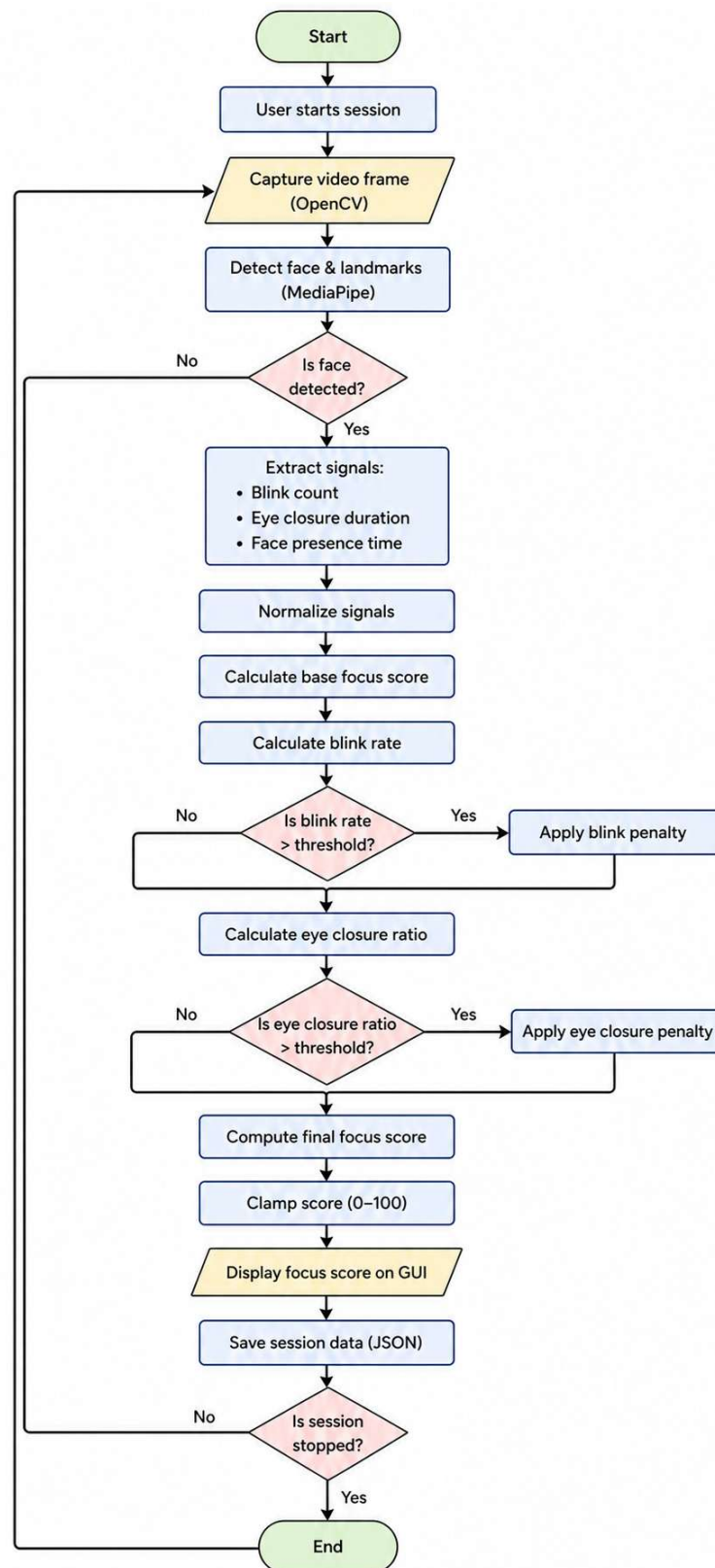
Level 1



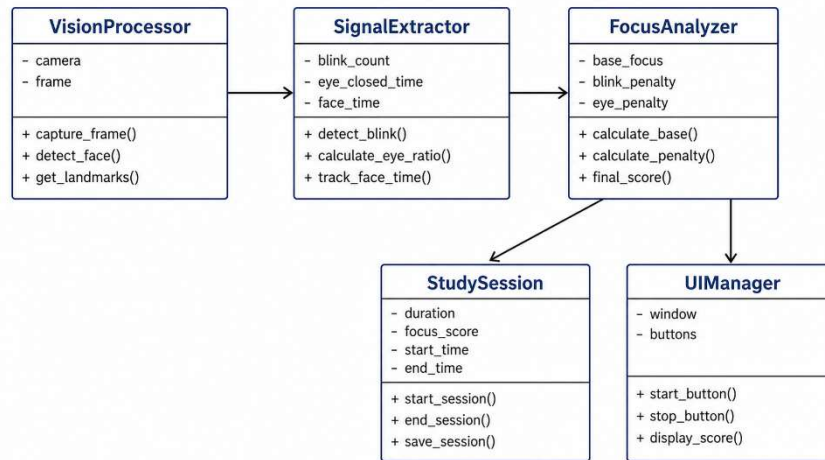
3.3.2 Activity Diagram



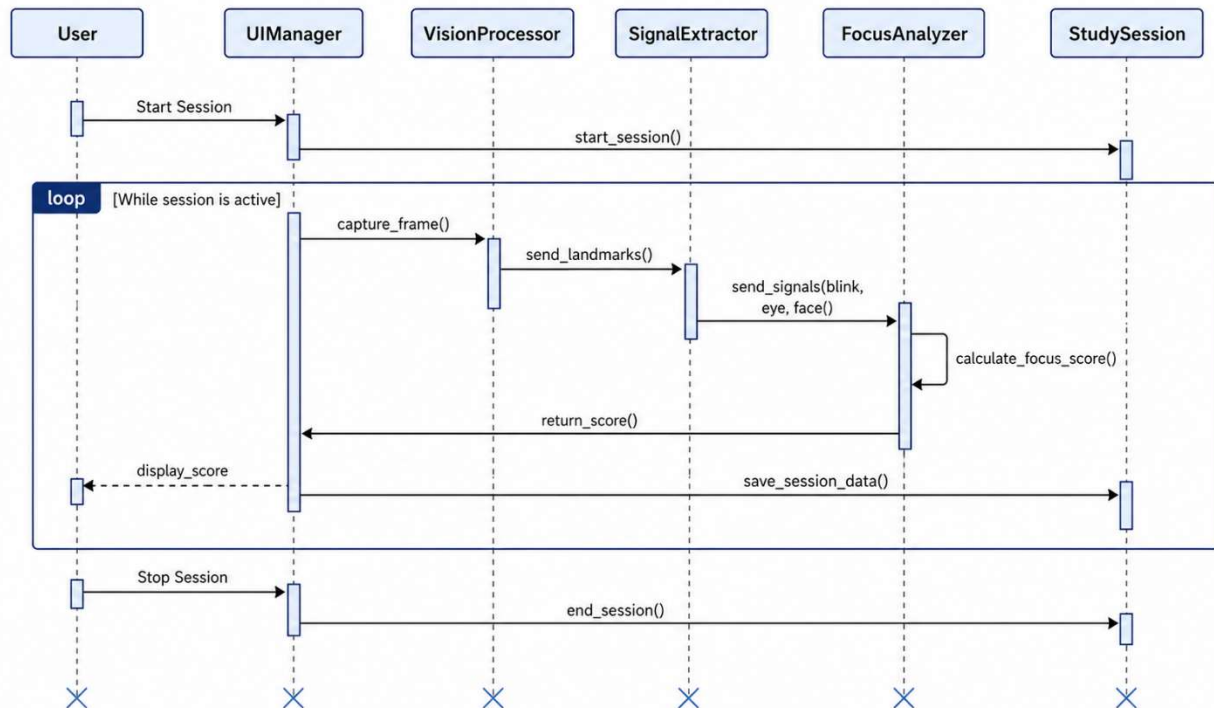
3.3.3 Flow Chart



3.3.4 Class Diagram



3.3.5 Sequence diagram



3.4 Database Design

The system uses a lightweight data storage approach instead of a traditional database management system. Since the application is designed to operate offline and does not require complex data handling, a file-based storage method using JSON (JavaScript Object Notation) is implemented.

JSON is chosen due to its simplicity, readability, and ease of integration with Python. It allows structured storage of session-related data without the need for database setup or maintenance.

The system stores data locally after each session, including key information such as session duration and focus score. This enables users to review past performance and track improvements over time.

The database design is divided into logical and physical representations as described below.

3.4.1 Logical Database Design

The logical design defines how data is organized conceptually.

The system stores session-related information in a structured format. Each session is represented as an individual record containing the following fields:

- i. Session ID
- ii. Start Time
- iii. End Time
- iv. Session Duration
- v. Focus Score

Each record is independent and represents a single user session. The data is organized as a collection of such records, allowing easy access and modification.

The logical structure ensures that data is simple, consistent, and easy to interpret.

3.4.2 Physical Database Design

The physical design describes how data is actually stored in the system.

The data is stored in a JSON file named:

“sessions.json”

The structure of the JSON file is as follows:

```
[  
  {
```

```
"session_id": 1,  
"start_time": "2024-01-01 10:00:00",  
"end_time": "2024-01-01 10:30:00",  
"duration": 30,  
"focus_score": 78  
}  
]
```

Each object in the JSON array represents a session record.

The file is stored locally within the system directory, and data is appended after each session. This approach ensures minimal storage overhead while maintaining structured and accessible data.

Chapter 4

4.1 Introduction to Languages, IDEs, Tools and Technologies Used for Implementation

The development of the proposed system involves the use of various programming languages, tools, and libraries that enable efficient implementation of computer vision and graphical user interface components.

The key technologies used in the project are as follows:

Python

Python is used as the primary programming language due to its simplicity, readability, and strong support for libraries related to computer vision and data processing.

OpenCV

OpenCV is used for capturing real-time video input from the webcam and performing basic image processing operations required for frame handling.

MediaPipe

MediaPipe is used for detecting facial landmarks, especially eye and face regions. It provides accurate and real-time tracking without requiring complex model training.

NumPy

NumPy is used for performing numerical calculations and processing signals such as blink ratios and eye closure measurements.

Tkinter

Tkinter is used for developing the graphical user interface. It provides an interactive platform for users to start and stop sessions and view results.

JSON

JSON is used for storing session data locally in a structured and readable format.

Development Tools

Integrated Development Environment (IDE):

The project is developed using an IDE such as Visual Studio Code or PyCharm, which provides features like code editing, debugging, and project management.

Python Package Manager (pip):

Used for installing and managing required libraries such as OpenCV, MediaPipe, and NumPy.

The combination of these technologies ensures that the system is efficient, lightweight, and capable of performing real-time focus analysis without requiring complex infrastructure.

4.2 Testing Techniques and Test Plans

Testing is an essential phase of the system development process to ensure that the application functions correctly, efficiently, and without errors. The system is tested using different techniques to validate its performance and reliability.

Testing Techniques

Unit Testing

Each module of the system, such as video capture, signal extraction, and focus analysis, is tested individually to ensure correct functionality.

Integration Testing

After individual modules are tested, they are combined and tested together to verify that data flows correctly between components.

System Testing

The entire system is tested as a whole to ensure that all functionalities work together as expected.

User Interface Testing

The graphical interface is tested to ensure that buttons, labels, and display elements function correctly and respond to user actions.

Performance Testing

The system is tested under continuous operation to ensure stability, responsiveness, and efficient resource usage.

Test Plan

The following test cases are used to validate system performance:

Test Case 1: Session Start and Stop

Verify that the system correctly starts and stops a session when the user interacts with the interface.

Test Case 2: Face Detection

Ensure that the system detects the user's face accurately under normal lighting conditions.

Test Case 3: Blink Detection

Check whether the system correctly counts eye blinks during the session.

Test Case 4: Eye Closure Detection

Verify that the system measures the duration of eye closure accurately.

Test Case 5: Focus Score Calculation

Ensure that the focus score is calculated correctly based on input signals.

Test Case 6: Real-Time Display

Check that the focus score and session time are updated continuously without delay.

Test Case 7: Data Storage

Verify that session data is correctly stored in the JSON file after completion.

4.3 Installation Instructions

The following steps should be followed to install and run the system on a local machine:

Step 1: Install Python

Download and install Python (version 3.8 or higher) from the official website:

<https://www.python.org>

Ensure that Python is added to the system PATH during installation.

Step 2: Set Up Project Directory

Download or clone the project files into a local directory.

Open the project folder using an IDE such as Visual Studio Code or PyCharm.

Step 3: Install Required Libraries

Open the terminal in the project directory and run the following commands:

```
pip install opencv-python
```

```
pip install mediapipe
```

```
pip install numpy
```

Tkinter is usually pre-installed with Python. If not, install it separately.

Step 4: Verify Webcam

Ensure that the system has a working webcam.
Grant necessary permissions if required.

Step 5: Run the Application

Open the main file (main.py) in the project directory.

Run the following command:

```
python main.py
```

Step 6: Launch Application

Click on "Start Session" to begin focus analysis.

Step 7: Stop Session

Click on "Stop Session" to end the session and store results.

4.4 End User Instructions

The system is designed to be simple and user-friendly. The following steps guide the user on how to operate the application effectively:

Steps to Use the System

Step 1: Launch the Application

Run the main program file to open the graphical user interface.

Step 2: Position Yourself

Sit in front of the webcam at an appropriate distance with proper lighting to ensure accurate detection.

Step 3: Start Session

Click on the "Start Session" button to begin focus analysis.

Step 4: Maintain Focus

Continue your study or work while the system monitors your attention using facial and eye movements.

Step 5: Monitor Output

Observe the real-time display of:

- i. Focus score
- ii. Session duration

Step 6: Stop Session

Click on the “Stop Session” button to end the session.

Step 7: View Results

After stopping the session, the final focus score is displayed and stored for future reference.

Chapter 5

5.1 User Interface Representation

The system provides a simple and user-friendly graphical user interface developed using Tkinter. The interface is designed to allow users to interact with the system easily and monitor their focus in real time.

The main components of the user interface include:

Application Window

The main window displays all controls and information related to the system in a structured layout.

Start Session Button

Allows the user to begin the focus analysis process. Once clicked, the system starts capturing video and analyzing attention.

Stop Session Button

Stops the ongoing session and ends the analysis. The final results are displayed and stored.

Focus Score Display

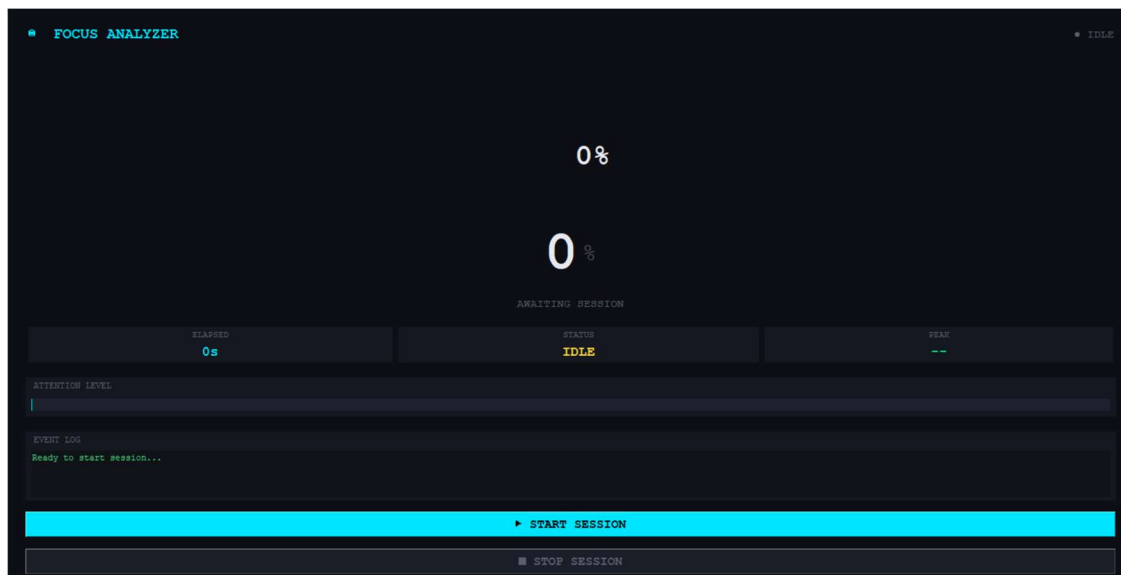
Shows the current focus score in real time, helping the user understand their level of attention.

Session Timer

Displays the duration of the ongoing session, indicating how long the user has been active.

Status Indicator

Shows whether the system is running or stopped.



5.2 Brief Description of Various Modules of the System

The system is divided into multiple modules, each responsible for a specific function. This modular design improves system organization, maintainability, and efficiency.

The major modules of the system are described below:

Video Capture Module

This module is responsible for capturing real-time video input from the webcam using OpenCV. It continuously retrieves frames for further processing.

Face and Landmark Detection Module

This module uses MediaPipe to detect the user's face and extract facial landmarks. It focuses on identifying key regions such as the eyes, which are essential for attention analysis.

Signal Extraction Module

This module processes the detected landmarks to extract behavioral signals, including blink count, eye-closure duration, and face presence time.

Focus Analysis Module

This module applies a rule-based approach to calculate the focus score. It normalizes the signals, applies penalties, and computes the final score.

User Interface Module

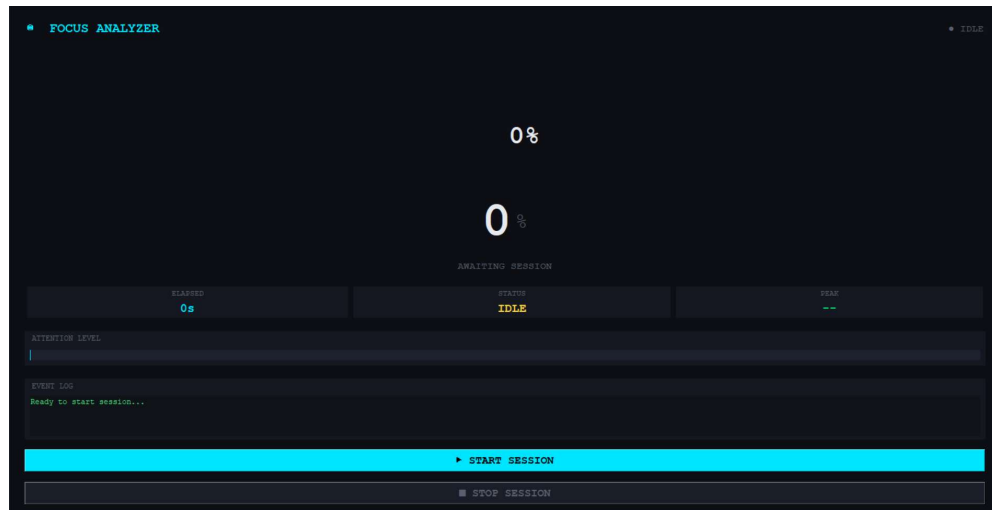
Developed using Tkinter, this module provides an interface for user interaction. It includes controls for starting and stopping sessions and displays real-time results.

Data Storage Module

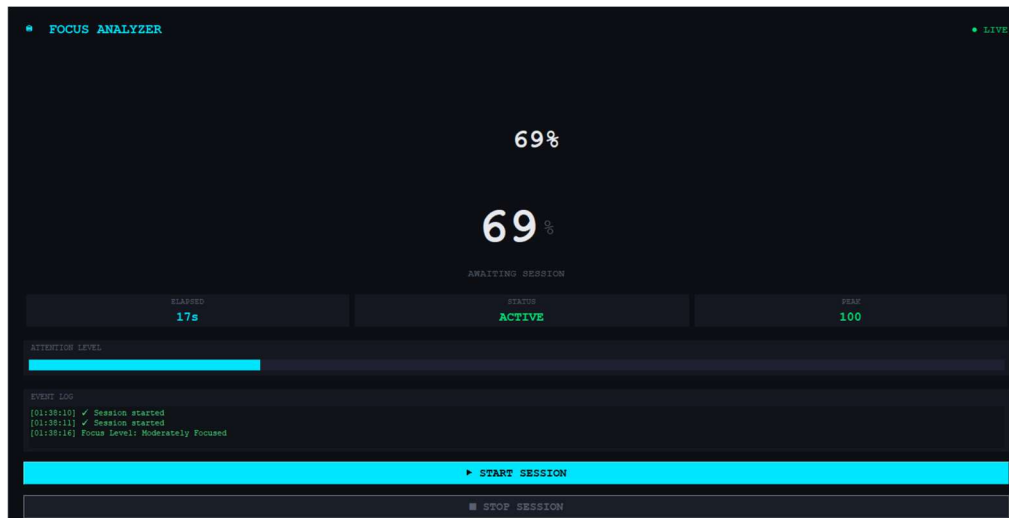
This module stores session data such as focus score and session duration in a JSON file for future reference and analysis.

5.3 Snapshots of System with Brief Detail

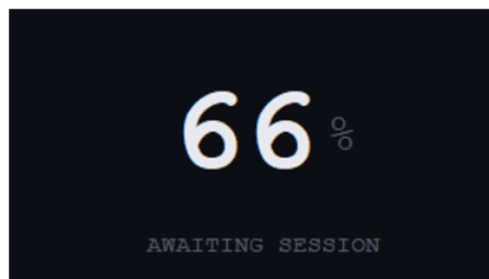
5.3.1: Initial Interface



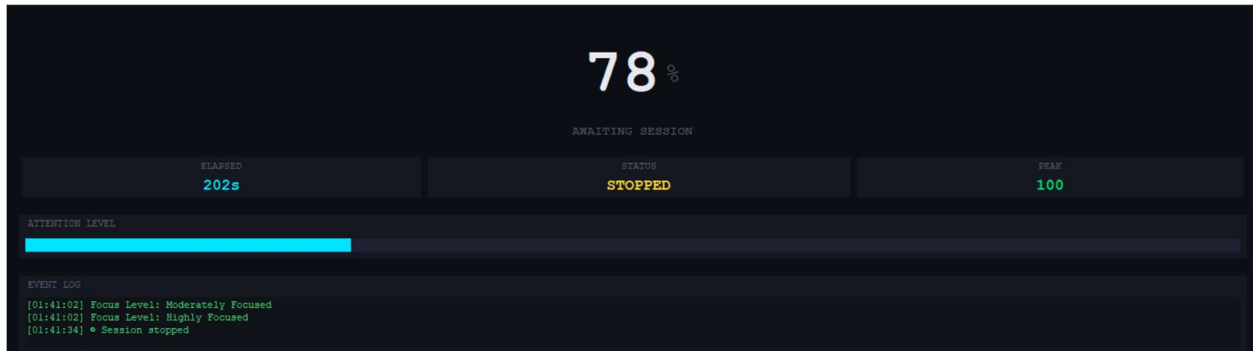
5.3.2: Session in Progress



5.3.3: Focus Score Display



5.3.4: Session Completed



5.4 Back End Representation

The backend of the system is designed to handle data processing, analysis, and storage efficiently. Unlike traditional applications that rely on complex database management systems, this project uses a lightweight file-based storage approach.

The system processes data in real time using Python and its associated libraries. The backend is responsible for capturing video input, extracting behavioral signals, computing the focus score, and storing session data.

The key components of the backend include:

Data Processing Engine

Handles real-time video processing using OpenCV and MediaPipe to extract facial landmarks and behavioral signals.

Focus Analysis Logic

Implements the rule-based algorithm to calculate the focus score based on extracted signals such as blink rate, eye-closure duration, and face presence.

Data Storage Mechanism

Stores session data locally in a JSON file, ensuring simple and structured data management.

Backend Data Structure

The session data is stored in a JSON file with the following structure:

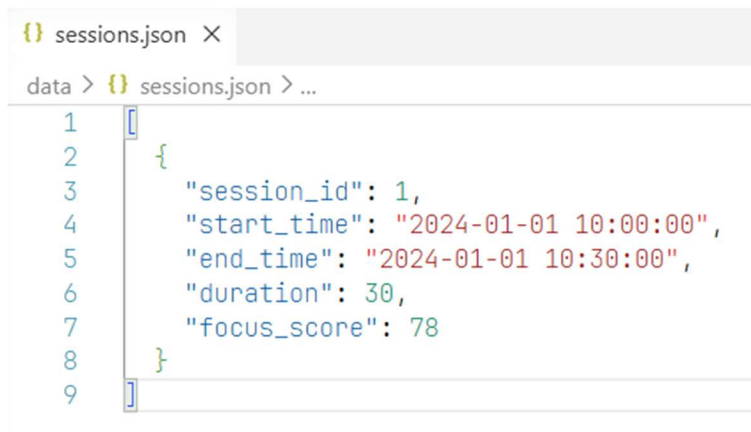
```
[
  {
    "session_id": 1,
    "start_time": "2024-01-01 10:00:00",
```

```
"end_time": "2024-01-01 10:30:00",  
"duration": 30,  
"focus_score": 78  
}  
]
```

5.5 Snapshots of Database Tables with Brief Description

Since the system uses a JSON-based storage approach instead of a traditional database, the stored data is represented in a structured file format rather than relational tables.

5.5.1: JSON Data Storage



```
{  
  "session_id": 1,  
  "start_time": "2024-01-01 10:00:00",  
  "end_time": "2024-01-01 10:30:00",  
  "duration": 30,  
  "focus_score": 78  
}
```


Chapter 6: Summary and Conclusions

The project titled “**AI Based Focus and Attention Analysis System**” presents a practical and efficient solution for analyzing student attention during study sessions using computer vision techniques. The system successfully addresses the limitation of traditional methods that rely only on time tracking by introducing a mechanism to evaluate the quality of focus.

The system utilizes real-time video input to detect facial and eye-related behavioral signals such as blink frequency, eye-closure duration, and face presence. These signals are processed using a rule-based approach to generate a focus score ranging from 0 to 100. The implementation ensures that the system remains lightweight, explainable, and easy to use without requiring complex machine learning models or large datasets.

The use of technologies such as OpenCV and MediaPipe enables accurate and efficient detection, while the Tkinter-based graphical interface provides a simple platform for user interaction. The system also ensures privacy by operating entirely offline and storing data locally in JSON format.

The modular design of the system allows for clear separation of functionalities, making it easier to develop, test, and maintain. Each module performs a specific role, contributing to the overall performance and reliability of the system.

In conclusion, the project demonstrates that it is possible to build an effective attention analysis system using simple and efficient techniques. The system not only provides real-time feedback but also encourages users to improve their study habits and productivity by increasing awareness of their focus patterns.

Chapter 7: Future Scope

The proposed AI Based Focus and Attention Analysis System provides a strong foundation for analyzing user attention using computer vision techniques; however, there are several opportunities for further enhancement and expansion. In the future, advanced machine learning or deep learning models can be integrated to improve the accuracy of attention prediction by learning more complex behavioral patterns. Additional features such as gaze tracking and head pose estimation can be incorporated to provide a more comprehensive analysis of user engagement. The system can also be extended to mobile platforms to increase accessibility and usability. Implementing cloud-based storage would allow long-term tracking and comparison of user performance across multiple sessions and devices. Furthermore, a personalized feedback system can be developed to provide suggestions based on user behavior, along with graphical reports to visualize focus trends over time. The system can also be expanded to support multiple users and integrated with online learning platforms for real-time monitoring during virtual classes. These enhancements would make the system more intelligent, scalable, and applicable in a wider range of educational and professional environments.

Bibliography

[1]OpenCV Documentation.

Available at: <https://opencv.org>

[2]MediaPipe Face Mesh Documentation.

Available at: <https://developers.google.com/mediapipe>

[3]Python Software Foundation. *Python Documentation*.

Available at: <https://www.python.org>

[4]NumPy Documentation.

Available at: <https://numpy.org>

[5]Tkinter Documentation (Python GUI Library).

Available at: <https://docs.python.org/3/library/tkinter.html>

[6]Rosebrock, A. *Practical Python and OpenCV*. PyImageSearch, 2016.

[7]Szeliski, R. *Computer Vision: Algorithms and Applications*. Springer, 2010.

[8]Bradski, G., and Kaehler, A. *Learning OpenCV: Computer Vision with the OpenCV Library*. O'Reilly Media, 2008.